FICHA PRÁTICA LABORATORIAL 02 – AI-POWERED SOFTWARE ENGINEERING

Objetivos

- Aplicar IA na definição de uma arquitetura adequada ao MVP da Matriz.
- Identificar entidades e relacionamentos e gerar um diagrama ER em Mermaid.
- Definir contratos REST simplificados com OpenAPI.
- Criar um wireframe de um ecrã da aplicação.
- Validar criticamente a consistência dos artefactos produzidos.

ENQUADRAMENTO DO PROJETO

Na Ficha Prática Laboratorial 01 foi elaborado o **Specification Pack** da **Matriz**, uma aplicação de gestão de projetos e tarefas para equipas de trabalho.

Nesta atividade, os requisitos e as User Stories já definidos servirão de base à conceção da solução: arquitetura, modelo de dados, contratos de API e protótipo de interface.

Importante: Não deverá ser implementado código funcional. O foco está na produção de um Architecture Pack simplificado, adequado ao MVP e preparado para reutilização na Aula 03.

PREPARAÇÃO DO AMBIENTE

Abrir o repositório **Matriz** desenvolvido na Aula 01.
Disponibilizar à IA o contexto de docs/requirements.md, docs/user-stories.md, docs/constraints.md e AGENTS.md.
Selecionar uma ferramenta de IA e manter as decisões e os pressupostos separados dos requisitos confirmados.
Exercícios Práticos

Exercício 1 — Definição da Arquitetura

Objetivo: Escolher e justificar uma solução arquitetural para a Matriz.

Solicitar à IA uma comparação breve entre monólito, monólito modular e microservices, considerando o âmbito do MVP.
Selecionar a alternativa mais adequada e identificar os principais componentes e respetivas responsabilidades.
Registar a decisão e a justificação em docs/architecture.md.

Resultado esperado: Proposta arquitetural simples e justificada.

Exercício 2 — Modelação de Dados e Diagrama ER

Objetivo: Representar os dados essenciais da aplicação.

Analisar os requisitos do MVP e identificar entidades, atributos, identificadores e relacionamentos.

Definir cardinalidades e assinalar regras de negócio ou pressupostos por confirmar.

Gerar o diagrama ER em Mermaid (erDiagram) e rever a sua coerência.

Guardar o modelo resumido e o código Mermaid em docs/database.md.

Resultado esperado: Modelo de dados e diagrama ER coerentes com os requisitos.

Exercício 3 — API Design com OpenAPI

Objetivo: Documentar duas operações REST relevantes para o MVP.

Selecionar duas operações e definir os respetivos métodos HTTP e endpoints.

Identificar parâmetros, schemas de pedido/resposta e principais status codes.

Gerar uma especificação OpenAPI 3.1 em YAML e guardá-la em openapi.yaml.

Resultado esperado: Contrato OpenAPI simplificado com, pelo menos, duas operações documentadas.

Exercício 4 — Prototipagem de Interface

Objetivo: Representar visualmente um ecrã da Matriz.

Utilizar uma ferramenta de IA para gerar um wireframe de baixa fidelidade (por exemplo, o dashboard).

Validar se a navegação e os elementos apresentados correspondem às funcionalidades do MVP.

Guardar o wireframe em docs/ui/ ou registar o link para o protótipo.

Resultado esperado: Wireframe simples de um ecrã da aplicação.

Exercício 5 — Revisão Crítica e Human-in-the-Loop

Objetivo: Validar a consistência dos artefactos produzidos.

Pedir à IA para identificar omissões ou contradições entre requisitos, arquitetura, dados, API e wireframe.

Analisar criticamente as sugestões e corrigir os problemas relevantes, sem aceitar alterações automaticamente.

Resultado esperado: Architecture Pack simplificado e revisto.

ESTRUTURA FINAL DO PROJETO

No final da atividade, o repositório deverá conter, além dos ficheiros da Aula 01, os seguintes artefactos:

```
matriz/
├── README.md
├── AGENTS.md
├── openapi.yaml
└── docs/
    ├── requirements.md
    ├── user-stories.md
    ├── constraints.md
    ├── architecture.md
    ├── database.md
    └── ui/
        └── wireframe.png (ou link para o protótipo)
```

Os artefactos deverão estar organizados e manter consistência com o Specification Pack da Aula 01.

CRITÉRIOS DE VALIDAÇÃO

Antes de concluir a atividade, verificar:

- ☐ A arquitetura está justificada e adequada ao MVP.
- ☐ O modelo de dados inclui as principais entidades e relações.
- ☐ O diagrama ER em Mermaid está coerente com o modelo.
- ☐ O OpenAPI descreve, pelo menos, duas operações REST.
- ☐ Existe um wireframe de um ecrã relevante.
- ☐ Os pressupostos e as decisões pendentes estão identificados.
- ☐ Os artefactos foram revistos e versionados em Git.

ENTREGA

O resultado desta ficha será o Architecture Pack simplificado da Matriz, composto por docs/architecture.md, docs/database.md, openapi.yaml e um wireframe ou respetiva referência, versionados em Git.

Estes artefactos servirão de base à Aula 03 — Desenvolvimento Full-Stack com AI Assistants.

Bom trabalho! ☺